

ChatMessage 组件使用指南

ChatMessage 用来呈现聊天产品中的单条消息。它负责把接口返回的文本、结构化数据、Markdown、媒体和附件展示成稳定的消息气泡，并提供状态、流式输出、图片预览、附件点击等业务接口。

适用版本

scq-vue 1.0.16 及以上

框架要求

Vue 3.3+

文档日期

2026-06-21

1. [组件定位](#)
2. [安装与引入](#)
3. [快速上手](#)
4. [内容类型](#)
5. [头像、昵称和时间](#)
6. [流式输出](#)
7. [消息状态](#)
8. [图片与视频](#)
9. [文件附件](#)
10. [消息操作](#)
11. [样式定制](#)
12. [配置项、事件和类型](#)
13. [使用建议](#)

1. 组件定位

ChatMessage 只处理“单条消息”的展示和交互，不内置消息列表、滚动到底部、接口请求或会话状态管理。这样做的好处是边界清楚：业务侧可以自由组合消息列表、SSE、WebSocket、分页加载、消息撤回等逻辑。

AI / 用户气泡

Markdown

JSON

代码高亮

图片预览

视频播放

文件附件

流式光标

状态提示

操作插槽

典型使用场景：AI 助手、客服对话、知识库问答、工单沟通、审核记录、数据分析结果展示、带附件的消息流。

2. 安装与引入

2.1 安装

```
pnpm add scq-vue
# 或
npm install scq-vue
# 或
yarn add scq-vue
```

2.2 全局注册

适合中后台或组件使用频率较高的项目。样式可直接引入完整样式包。

```
import { createApp } from 'vue'
import ScqVue from 'scq-vue'
import 'scq-vue/style.css'
import App from './App.vue'

createApp(App).use(ScqVue).mount('#app')
```

```
<template>
  <scq-chat-message message="你好，我是 SCQ Assistant。" role="ai" />
</template>
```

2.3 按需引入

如果项目只用到 ChatMessage，可以按需引入组件和对应样式。

```
import { ChatMessage } from 'scq-vue'
import 'scq-vue/styles/chat-message.css'
```

```
<template>
  <ChatMessage :message="message" role="ai" />
</template>

<script setup lang="ts">
import { ChatMessage } from 'scq-vue'
import 'scq-vue/styles/chat-message.css'

const message = '这是一条来自接口的普通文本。'
</script>
```

样式说明：使用 `scq-vue/style.css` 会引入构建后的完整样式；使用 `scq-vue/styles/chat-message.css` 只引入 ChatMessage 的独立样式。团队可以根据打包策略选择其中一种。

3. 快速上手

最少只需要传入 `message`。`role` 决定气泡方向和视觉样式，默认是 `ai`。

```
<template>
  <scq-chat-message
    message="我已经整理好结果，你可以继续追问。"
    role="ai"
    name="SCQ Assistant"
    :timestamp="Date.now()"
  />

  <scq-chat-message
    message="帮我把结论压缩成三句话。"
    role="user"
    name="Chauncy"
    :show-time="false"
  />
</template>
```

如果要做消息列表，建议由外层组件负责循环、滚动和数据加载，ChatMessage 只负责渲染一条消息。

```
<template>
  <section class="message-list">
    <scq-chat-message
      v-for="item in messages"
      :key="item.id"
      :message="item.content"
      :role="item.role"
      :name="item.name"
      :timestamp="item.createdAt"
    />
  </section>
</template>
```

4. 内容类型

`contentType` 控制消息正文的渲染方式。默认值是 `auto`，组件会先识别媒体，再识别 JSON 和 Markdown，最后回退到普通文本。

类型	适合场景	说明
auto	接口返回内容不固定	自动识别图片、视频、JSON、Markdown 和纯文本。
text	普通聊天文本	按纯文本显示，保留换行，不解析 Markdown。
markdown	AI 回复、富文本说明、代码片段	支持标题、列表、链接、表格、代码块和自动换行。
json	调试接口、展示结构化结果	对象会格式化成缩进 JSON，字符串 JSON 会尝试美化。
image	图片消息、截图、生成图	支持懒加载、加载失败文案和点击预览。
video	视频消息、录屏、演示片段	支持 controls、poster、autoplay、muted、loop、preload。

4.1 普通文本

```
<scq-chat-message
  content-type="text"
  message="第一行\n第二行会保留换行。"
/>
```

4.2 Markdown 与代码高亮

适合 AI 输出、使用说明和带代码的回答。Markdown 链接默认添加 `target="\_blank"` 和 `rel="noopener noreferrer"`，也可以通过配置修改。

```
<scq-chat-message
  role="ai"
  content-type="markdown"
  :message="markdownText"
  link-target="_blank"
  link-rel="noopener noreferrer"
/>
```

```
const markdownText = `# 分析结果
```

- 已读取项目配置
- 已完成构建检查

```
\\\`ts  
const passed = true  
\\\`
```

4.3 JSON 数据

```
<scq-chat-message  
  role="ai"  
  content-type="json"  
  :message="apiResult"  
>
```

```
const apiResult = {  
  code: 0,  
  message: 'ok',  
  data: [{ id: 1, name: 'SCQ' }],  
}
```

5. 头像、昵称和时间

`avatar`、`name` 和 `timestamp` 用于补充消息来源。没有头像但传了昵称时，会显示昵称首字母头像。

```
<scq-chat-message  
  message="头像可以传图片，也可以只传昵称。"  
  role="ai"  
  name="SCQ Assistant"  
  avatar="https://example.com/avatar.png"  
  :timestamp="Date.now()"  
  :time-formatter="formatTime"  
>
```

```
const formatTime = (value: string | number | Date | null | undefined) => {  
  if (!value) return ''  
  const date = value instanceof Date ? value : new Date(value)  
  return date.toLocaleString('zh-CN', { hour12: false })  
}
```

需求	推荐配置
紧凑消息列表	`showAvatar=false`、`showName=false`、`showTime=false`
客服或多人会话	保留 `name` 和 `avatar`，便于区分角色。
消息时间格式统一	在外层封装 `timeFormatter`，不要在每条消息里写重复逻辑。

6. 流式输出

`streaming` 只负责显示流式光标，不负责建立 SSE 或 WebSocket 连接。业务侧持续更新 `message` 即可。

```
<template>
  <scq-chat-message
    :message="answer || '正在生成...'"
    role="ai"
    content-type="markdown"
    :streaming="streaming"
  />
</template>

<script setup lang="ts">
import { ref } from 'vue'

const answer = ref('')
const streaming = ref(false)

const start = () => {
  const source = new EventSource('/api/chat')
  streaming.value = true

  source.onmessage = (event) => {
    answer.value += event.data
  }

  source.onerror = () => {
    streaming.value = false
    source.close()
  }
}
</script>
```

行为说明： 图片和视频内容不会显示流式光标。它们通常是完整资源，不适合用文本光标表达“生成中”。

7. 消息状态

状态适合展示“正在思考”“正在调用工具”“处理完成”“失败”等消息级反馈。组件统一通过 `status` 对象配置状态；不传 `status` 时，不展示状态区域。

```
<scq-chat-message
  message="正在整理上下文，稍后给出完整回答。"
  role="ai"
  :status="{ type: 'thinking', text: '正在思考中' }"
/>

<scq-chat-message
  message="构建检查完成。"
  role="ai"
  :status="{ type: 'success', text: '处理完成', loading: false }"
/>
```

状态类型	适合场景
thinking	AI 正在组织回答、规划步骤。
tool	正在调用接口、工具、插件或外部系统。
loading	普通处理中状态。
success	任务已经完成。
warning	结果可用，但需要用户注意。
error	请求失败、工具失败或内容不可用。

8. 图片与视频

8.1 图片消息

图片可以直接传 URL，也可以传对象。`previewable` 控制是否允许点击预览，加载失败会显示 `imageFallbackText`。

```
<scq-chat-message
  :message="imageUrl"
  role="ai"
  content-type="image"
  image-fallback-text="图片暂时无法加载"
  @preview-open="handlePreviewOpen"
  @preview-close="handlePreviewClose"
  @image-error="handleImageError"
/>
```

```
const imageUrl = 'https://example.com/result.png'

const handlePreviewOpen = (src: string) => {
  console.log('打开预览', src)
}

const handlePreviewClose = (src: string) => {
  console.log('关闭预览', src)
}

const handleImageError = (src: string) => {
  console.warn('图片加载失败', src)
}
```

8.2 视频消息

```
<scq-chat-message
  role="user"
  content-type="video"
  :message="videoPayload"
/>
```

```
const videoPayload = {
  src: 'https://example.com/demo.mp4',
  poster: 'https://example.com/poster.png',
  controls: true,
  preload: 'metadata',
}
```

8.3 自动识别媒体

`contentType="auto"` 时，组件会识别对象里的 `type`、`mediaType`、`contentType`，也会根据 URL 后缀判断图片或视频。

```
const mediaMessage = {
  type: 'image',
  src: 'https://example.com/chart.webp',
  alt: '分析图表',
}
```

9. 文件附件

附件适合展示模型生成的文件、用户上传的材料、工单附件和下载资源。组件会根据文件名识别常见类型，也可以手动指定 `type`。

```
<scq-chat-message
  :message="attachmentMessage"
  role="ai"
  :attachments="fileAttachments"
  :attachment-click="handleAttachmentClick"
  @attachment-click="trackAttachmentClick"
/>
```

```
const attachmentMessage = '下面是本次分析用到的文件。'

const fileAttachments = [
  {
    name: '需求说明.pdf',
    url: 'https://example.com/spec.pdf',
    size: 245760,
    download: true,
  },
  {
    name: '数据分析.xlsx',
    url: 'https://example.com/report.xlsx',
    size: '1.2 MB',
  },
  {
    name: '上传中.zip',
    status: 'uploading',
    progress: 48,
    statusText: '上传中',
  },
]

const handleAttachmentClick = (payload, event) => {
  if (payload.status === 'uploading') {
    event.preventDefault()
    return false
  }
}

const trackAttachmentClick = (payload) => {
  console.log('附件点击', payload.name, payload.type)
}
```

附件状态	展示含义	常见用法
default	普通附件	可打开或下载。
uploading	上传中	配合 `progress` 显示进度，默认阻止打开。
success	上传或生成成功	用于明确任务结果。
error	失败	提示用户重新上传或重试。

10. 消息操作

通过 `actions` 插槽可以放置复制、重新生成、点赞、点踩、引用、删除等按钮。组件只提供容器样式，具体行为由业务决定。

```
<scq-chat-message message="这是一条可操作的回答。" role="ai">
  <template #actions>
    <button type="button" class="scq-chat-message__action" @click="copy">复制</button>
    <button type="button" class="scq-chat-message__action" @click="retry">重新生成</button>
    <button type="button" class="scq-chat-message__action" @click="feedback('up')">有帮助</button>
  </template>
</scq-chat-message>
```

建议：如果同一项目中所有消息都有相同操作，可以在业务侧再封装一层 `AppChatMessage`，把复制、重试、反馈等动作统一进去。

11. 样式定制

ChatMessage 的样式类名统一使用 `scq-chat-message` 前缀。项目里如果需要调整消息框外观，推荐在业务样式文件中覆盖这些类名，而不是直接改组件源码。这样后续升级组件时更容易合并。

11.1 推荐覆盖方式

全局样式文件中可以直接覆盖类名。若在 Vue 单文件组件的 `scoped` 样式里修改，需要使用 `:deep()` 穿透到组件内部。

```
/* src/styles/chat-theme.css */
.scq-chat-message.is-ai .scq-chat-message__bubble {
  background: #ffffff;
  border-color: #d7e2f0;
  color: #1f2937;
}

.scq-chat-message.is-user .scq-chat-message__bubble {
  background: #1677ff;
  border-color: #1677ff;
  color: #ffffff;
}

.scq-chat-message__content {
  max-width: min(86%, 760px);
}
```

```
<style scoped>
.chat-page :deep(.scq-chat-message.is-ai .scq-chat-message__bubble) {
  background: #ffffff;
}

.chat-page :deep(.scq-chat-message__avatar) {
  width: 40px;
  height: 40px;
  flex-basis: 40px;
}
</style>
```

11.2 常用样式入口

类名	可调整内容	常见场景
.scq-chat-message	单条消息的外层间距、整体布局。	调整消息之间的距离。
.scq-chat-message.is-ai	AI 消息整体方向和派生样式。	给助手消息设置统一主题。
.scq-chat-message.is-user	用户消息整体方向和派生样式。	修改用户气泡颜色或对齐方式。
.scq-chat-message__body	头像和内容区的横向布局。	调整头像与气泡的间距。
.scq-chat-message__content	消息内容最大宽度、内部排列。	让气泡更宽或更窄。
.scq-chat-message__bubble	气泡背景、边框、圆角、阴影、字号、内边距。	定制消息框视觉风格。
.scq-chat-message__avatar	头像尺寸、边框、背景。	适配项目头像尺寸规范。
.scq-chat-message__name	昵称字号、颜色、间距。	多人会话或客服场景。
.scq-chat-message__time	时间颜色、字号、位置。	弱化时间信息或贴气泡。
.scq-chat-message__markdown	Markdown 段落、表格、链接、代码样式。	让 AI 输出符合项目内容样式。
.scq-chat-message__code-block	代码块背景、文字颜色、滚动条、圆角。	适配深色或浅色代码主题。
.scq-chat-message__attachments	附件列表宽度和间距。	调整附件卡片展示密度。
.scq-chat-message__attachment	附件卡片背景、边框、悬停效果。	贴合项目文件卡片风格。
.scq-chat-message__actions	操作区布局和间距。	放置复制、重试、反馈按钮。

11.3 场景示例

扩大消息框宽度

```
.scq-chat-message__content {  
  max-width: min(92%, 860px);  
}
```

改成更圆润的气泡

```
.scq-chat-message__bubble {  
  border-radius: 18px;  
  padding: 14px 16px;  
}  
  
.scq-chat-message.is-ai .scq-chat-message__bubble,  
.scq-chat-message.is-user .scq-chat-message__bubble {  
  border-bottom-left-radius: 18px;  
  border-bottom-right-radius: 18px;  
}
```

改用户消息颜色

```
.scq-chat-message.is-user .scq-chat-message__bubble {  
  background: #0f766e;  
  border-color: #0f766e;  
}  
  
.scq-chat-message.is-user .scq-chat-message__avatar {  
  background: #0f766e;  
}
```

调整 Markdown 表格和代码块

```
.scq-chat-message__markdown table {  
  font-size: 13px;  
}  
  
.scq-chat-message__markdown th {  
  background: #eef6ff;  
}  
  
.scq-chat-message__code-block {  
  background: #111827;  
  border-radius: 8px;  
}
```

维护建议：如果一个项目有统一的聊天主题，可以新建 `chat-theme.css`，集中覆盖 ChatMessage 样式，再在应用入口引入它。不要把样式分散写在多个页面里，后续会更难排查。

12. 配置项、事件和类型

12.1 Props

参数	类型	默认值	说明
message	unknown	-	消息内容。可以是字符串、对象、媒体对象等。
role	'ai' 'user'	'ai'	消息角色，决定对齐方向和气泡视觉。
contentType	'auto' 'text' 'markdown' 'json' 'image' 'video'	'auto'	正文渲染方式。
showTime	boolean	true	是否显示时间。没有 timestamp 或 formatter 返回空字符串时不会显示。
timestamp	string number Date null	null	消息时间。
timeFormatter	(value) => string	-	自定义时间格式化函数。
streaming	boolean	false	是否显示流式输出光标。
status	ChatMessageStatus null	null	消息状态对象。不传时不展示状态区域。
avatar	string	"	头像图片地址。
name	string	"	消息昵称。未传 avatar 时可生成首字母头像。
showAvatar	boolean	true	是否展示头像。
showName	boolean	true	是否展示昵称。
linkTarget	string	'_blank'	Markdown 链接 target。传空字符串可不设置。
linkRel	string	'noopener noreferrer'	Markdown 链接 rel。传空字符串可不设置。
previewable	boolean	true	图片是否支持点击预览。

参数	类型	默认值	说明
imageFallbackText	string	'图片加载失败'	图片加载失败时显示的文案。
attachments	ChatAttachment[]	[]	消息附件列表。
attachmentClick	(payload, event) => boolean void	-	附件点击回调。返回 false 可阻止默认打开或下载。

12.2 Events

事件	参数	触发时机
attachment-click	(payload, nativeEvent)	点击附件时触发。
preview-open	(src)	图片预览打开时触发。
preview-close	(src)	图片预览关闭时触发，包括点击遮罩、点击关闭按钮和按 Esc。
image-error	(src)	图片加载失败时触发。

12.3 Slots

插槽	说明
actions	消息操作区。适合放复制、重试、反馈、引用、删除等按钮。

12.4 TypeScript 类型

```
type ChatRole = 'ai' | 'user'
type ChatMediaType = 'image' | 'video'
type ChatContentType = 'auto' | 'text' | 'markdown' | 'json' | ChatMediaType
type ChatStatusType = 'thinking' | 'tool' | 'loading' | 'success' | 'warning' | 'error'
type ChatVideoPreload = 'auto' | 'metadata' | 'none'
type ChatAttachmentType = 'pdf' | 'word' | 'excel' | 'ppt' | 'zip' | 'code' | 'text' | 'image' | 'video' | 'audio' | 'file'
type ChatAttachmentStatus = 'default' | 'uploading' | 'success' | 'error'
```

```

interface ChatMessageStatus {
  type?: ChatStatusType
  text?: string
  loading?: boolean
  visible?: boolean
}

interface ChatMediaMessage {
  type?: ChatMediaType
  mediaType?: ChatMediaType
  contentType?: ChatMediaType
  src?: string
  url?: string
  source?: string
  alt?: string
  title?: string
  poster?: string
  controls?: boolean
  autoplay?: boolean
  muted?: boolean
  loop?: boolean
  preload?: ChatVideoPreload
}

interface ChatAttachment {
  name: string
  url?: string
  type?: ChatAttachmentType | string
  size?: number | string
  download?: boolean | string
  target?: string
  rel?: string
  status?: ChatAttachmentStatus
  statusText?: string
  progress?: number
}

```

13. 使用建议

- 消息列表、滚动到底部、接口请求和重试队列放在外层组件里管理，ChatMessage 保持单条消息职责。
- Markdown 内容来自用户输入时，当前组件默认关闭 HTML 解析；如果业务要放开 HTML，需要另外做安全审核。
- 附件点击如果要接入权限校验、预览系统或下载签名，优先使用 `attachmentClick` 接管默认行为。

- 图片预览事件适合做埋点、暂停页面快捷键、恢复滚动锁定等工作。
- 复杂项目建议封装业务版消息组件，把当前用户信息、统一时间格式、默认操作区、埋点等逻辑收进去。

注意： ChatMessage 不是聊天 SDK。它不负责网络连接、消息协议、登录态和会话持久化。把这些职责留在业务层，组件会更容易维护，也更方便在不同产品里复用。

更多组件示例和在线演示可参考：<https://shichangqi.github.io/scq-vue/#/guide>